All Codes + Pseudocode Cheat Sheet

Fractional Knapsack

```
n,w=map(int,input().split())
a=list(map(int,input().split()))
it=sorted([(a[i]/a[i+1],a[i],a[i+1])for i in range(0,2*n,2)],reverse=True)
ans=0
for r,v,wt in it:
 t=min(w,wt);ans+=r*t;w-=t
print(f"{ans:.6f}")
```

0/1 Knapsack

```
n=int(input());W=int(input())
val=list(map(int,input().split()))
wt=list(map(int,input().split()))
dp=[0]*(W+1)
for i in range(n):
 for w in range(W,wt[i]-1,-1):
  dp[w]=max(dp[w],dp[w-wt[i]]+val[i])
print(dp[W])
```

Knapsack With k

```
n,W,k=map(int,input().split())
dp=[[0]*(W+1)for _ in range(k+1)]
for _ in range(n):
 wt,val=map(int,input().split())
 for c in range(k,0,-1):
  for w in range(W,wt-1,-1):
   dp[c][w]=max(dp[c][w],dp[c-1][w-wt]+val)
print(max(map(max,dp)))
```

Max Profit 2 Txn

```
n=int(input())
a=list(map(int,input().split()))
b1=b2=float('inf');p1=p2=0
for x in a:
 b1=min(b1,x);p1=max(p1,x-b1)
 b2=min(b2,x-p1);p2=max(p2,x-b2)
print(p2)
```

Optimal BST

```
n=int(input());input()
f=list(map(int,input().split()))
pre=[0]
for x in f:pre.append(pre[-1]+x)
dp=[[0]*n for _ in range(n)]
for i in range(n):dp[i][i]=f[i]
for l in range(2,n+1):
 for i in range(n-l+1):
  j=i+l-1;s=pre[j+1]-pre[i]
  dp[i][j]=min((dp[i][r-1]if r>i else 0)+(dp[r+1][j]if r<j else 0) for r in range(i,j+1))+s
print(dp[0][n-1])
```

MCM

```
n=int(input())
a=list(map(int,input().split()))
dp=[[0]*n for _ in range(n)]
for l in range(2,n):
 for i in range(1,n-l+1):
  j=i+l-1
  dp[i][j]=min(dp[i][k]+dp[k+1][j]+a[i-1]*a[k]*a[j] for k in range(i,j))
print(dp[1][n-1])
```

OBST Depth Limit

```
from functools import lru_cache
n,D=map(int,input().split());input()
f=list(map(int,input().split()))
pre=[0]
for x in f:pre.append(pre[-1]+x)
@lru_cache(None)
def dp(i,j,d):
 if i>j:return 0
 if d==0 or j-i+1>(1<<d)-1:return 10**18
 s=pre[j+1]-pre[i]
 return min(dp(i,r-1,d-1)+dp(r+1,j,d-1)+s for r in range(i,j+1))
ans=dp(0,n-1,D);print(ans if ans<10**18 else -1)
```

LCS 3 Strings

```
x=input();y=input();z=input()
a,b,c=len(x),len(y),len(z)
dp=[[[0]*(c+1)for _ in range(b+1)]for _ in range(a+1)]
for i in range(1,a+1):
 for j in range(1,b+1):
  for k in range(1,c+1):
   dp[i][j][k]=dp[i-1][j-1][k-1]+1 if x[i-1]==y[j-1]==z[k-1] else max(dp[i-1][j][k],dp[i][j-1][k],dp[
i][j][k-1])
print(dp[a][b][c])
```

Floyd Warshall

```
n=int(input());e=int(input());INF=10**18
d=[[INF]*n for _ in range(n)]
for i in range(n):d[i][i]=0
for _ in range(e):
 s,t,w=map(int,input().split())
 d[s-1][t-1]=min(d[s-1][t-1],w)
for k in range(n):
 for i in range(n):
  for j in range(n):
   if d[i][k]!=INF and d[k][j]!=INF:d[i][j]=min(d[i][j],d[i][k]+d[k][j])
for r in d:
 for x in r:print("INF"if x==INF else x,end=" ")
 print()
```

Coin Change Ways

```
s=int(input());n=int(input())
coins=list(map(int,input().split()))
dp=[0]*(s+1);dp[0]=1
for c in coins:
 for i in range(c,s+1):
  dp[i]+=dp[i-c]
print(dp[s])
```

N Queens All

```
n=int(input());a=[];c=set();d1=set();d2=set();ok=0
def f(col):
 global ok
 if col==n:print(*a);ok=1;return
 for r in range(n):
  if r not in c and r-col not in d1 and r+col not in d2:
   a.append(r);c.add(r);d1.add(r-col);d2.add(r+col)
   f(col+1)
   a.pop();c.remove(r);d1.remove(r-col);d2.remove(r+col)
f(0)
if not ok:print(-1)
```

N Queens Board

```
n=int(input())
b=[[0]*n for _ in range(n)];c=set();d1=set();d2=set()
def f(r):
 if r==n:return 1
 for x in range(n):
  if x not in c and r-x not in d1 and r+x not in d2:
   b[r][x]=1;c.add(x);d1.add(r-x);d2.add(r+x)
   if f(r+1):return 1
   b[r][x]=0;c.remove(x);d1.remove(r-x);d2.remove(r+x)
for row in b if f(0) else b:print("["+", ".join(map(str,row))+"]")
```

Subset Sum

```
n,s=map(int,input().split())
a=list(map(int,input().split()));cur=[];ok=0
def f(i,sm):
 global ok
 if sm==s:print("["+" ".join(map(str,cur))+"]");ok=1;return
 if i==n or sm>s:return
 cur.append(a[i]);f(i+1,sm+a[i]);cur.pop();f(i+1,sm)
f(0,0)
if not ok:print("No subset found")
```

Graph Coloring

```
n,m=map(int,input().split())
g=[list(map(int,input().split())) for _ in range(n)];c=[0]*n
def ok(v,x):
 for i in range(n):
  if g[v][i] and c[i]==x:return 0
 return 1
def f(v):
 if v==n:
  for x in c:print(x,end=" ")
  return 1
 for x in range(1,m+1):
  if ok(v,x):
   c[v]=x
   if f(v+1):return 1
   c[v]=0
f(0)
```

Hamiltonian Cycle

```
n=int(input())
g=[list(map(int,input().split())) for _ in range(n)]
path=[0];vis=[0]*n;vis[0]=1
def f(v):
 if len(path)==n:
  if g[v][0]:print(*path,0);return 1
  return 0
 for u in range(1,n):
  if g[v][u] and not vis[u]:
   vis[u]=1;path.append(u)
   if f(u):return 1
   path.pop();vis[u]=0
if not f(0):print("Solution does not exist")
```

FW Matrix -1

```
n=int(input());INF=10**18
d=[list(map(int,input().split())) for _ in range(n)]
for i in range(n):
 for j in range(n):
  if i!=j and d[i][j]==-1:d[i][j]=INF
for k in range(n):
 for i in range(n):
  for j in range(n):
   if d[i][k]!=INF and d[k][j]!=INF:d[i][j]=min(d[i][j],d[i][k]+d[k][j])
for r in d:
 for x in r:print("INF"if x==INF else x,end=" ")
 print()
```

MSIS

```
n=int(input())
a=list(map(int,input().split()))
dp=a[:]
for i in range(n):
 for j in range(i):
  if a[j]<a[i]:dp[i]=max(dp[i],dp[j]+a[i])
print(max(dp))
```

Word Break

```
n=int(input());d=set(input().split());s=input()
dp=[0]*(len(s)+1);dp[0]=1
for i in range(1,len(s)+1):
 for j in range(i):
  if dp[j] and s[j:i] in d:
   dp[i]=1;break
print(dp[-1])
```

Sum of Subsets k

```
n,s,k=map(int,input().split())
a=list(map(int,input().split()));cur=[]
def f(i,sm):
 if sm==s:print("["+" ".join(map(str,cur))+"]");return
 if i==n or sm>s or len(cur)==k:return
 cur.append(a[i]);f(i+1,sm+a[i]);cur.pop();f(i+1,sm)
f(0,0)
```

Distinct Permutations

```
from itertools import permutations
s=input().strip()
print(*sorted(set(''.join(p) for p in permutations(s))))
```

Rod Cutting Order

```
from functools import lru_cache
N=int(input());M=int(input())
a=[0]+sorted(map(int,input().split()))+[N]
@lru_cache(None)
def dp(i,j):
 if j-i==1:return 0,()
 best=(10**18,())
 for k in range(i+1,j):
  c1,s1=dp(i,k);c2,s2=dp(k,j)
  cur=(a[j]-a[i])+c1+c2;seq=(a[k],)+s1+s2
  if cur<best[0] or (cur==best[0] and seq<best[1]):best=(cur,seq)
 return best
for x in dp(0,M+1)[1]:print(x,end=" ")
```

All Codes + Pseudocode Cheat Sheet

Pseudo - MCM

```
Algorithm MCM(arr,N)
Create dp[N][N]
For i=1 to N-1: dp[i][i]=0
For len=2 to N-1:
 For i=1 to N-len:
  j=i+len-1; dp[i][j]=INF
  For k=i to j-1:
   cost=dp[i][k]+dp[k+1][j]+arr[i-1]*arr[k]*arr[j]
   If cost<dp[i][j]: dp[i][j]=cost
Return dp[1][N-1]
```

Pseudo - 0/1 Knapsack

```
Algorithm Knapsack(W,wt,val,N)
Create dp[0..N][0..W]
For i=0 to N:
 For w=0 to W:
  If i=0 or w=0: dp[i][w]=0
  Else If wt[i]<=w:
   dp[i][w]=max(dp[i-1][w],val[i]+dp[i-1][w-wt[i]])
  Else: dp[i][w]=dp[i-1][w]
Return dp[N][W]
```

Pseudo - Coin Change

```
Algorithm CoinChange(amount,coins,M)
Create dp[0..amount]
For i=0 to amount: dp[i]=INF
dp[0]=0
For i=1 to amount:
 For j=1 to M:
  If coins[j]<=i and dp[i-coins[j]]!=INF:
   dp[i]=min(dp[i],dp[i-coins[j]]+1)
If dp[amount]=INF: Print "Not possible"
Else: Print dp[amount]
```